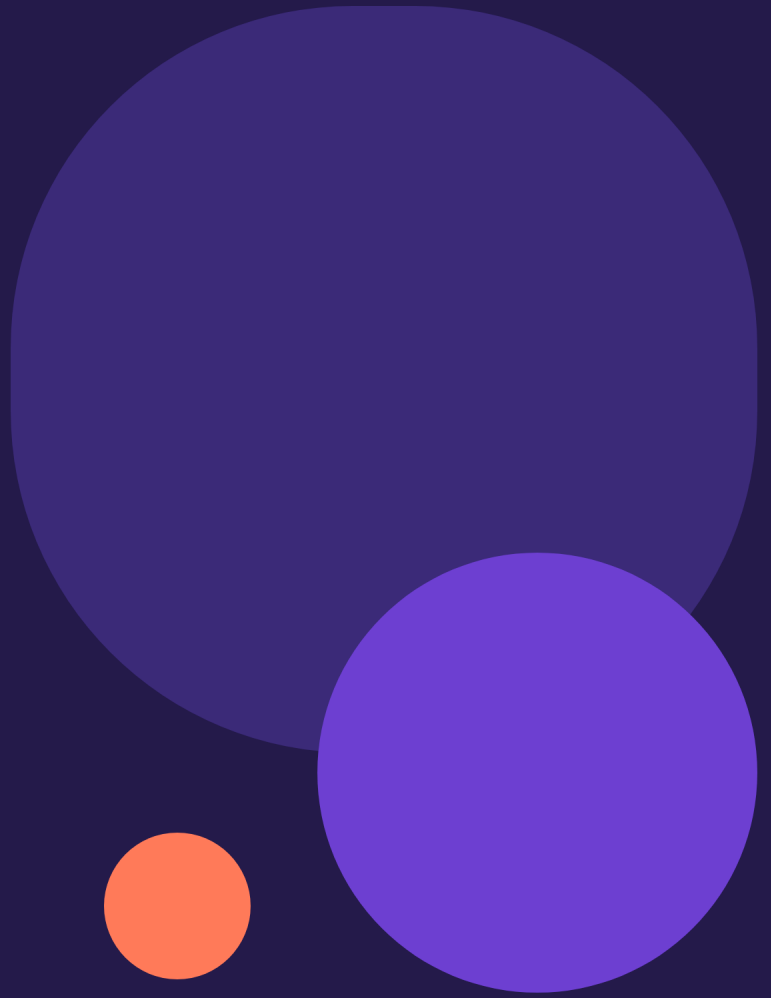
MDN WEB DOCS · SERVER SIDE

Sunucu Tarafı Web Programlama

Temel kavramlar · Django · Express · Ek eğitimler



Bu Derste Neler Öğreneceksin?

Dersin sonunda şunları yapabiliyor olmalısın:

Kavram	Sunucu ve istemci tarafını ayırt edebilecek, bir isteğin yolunu anlatabileceksin.
Çerçeve	Web çerçevesinin ne işe yaradığını ve Django ile Express'in farkını açıklayabileceksin.
Django	Model, URL, görünüm ve şablondan oluşan yapıyı tanıyıp basit bir sayfanın kodunu okuyabileceksin.
Express	Rota, denetleyici ve Mongoose şemasını tanıyıp aynı yapıyı Express'te okuyabileceksin.
Yayına alma	Test, güvenlik ve dağıtımın neden gerekli olduğunu söyleyebileceksin.

Sunucu Tarafı Programlama Nedir?

Dinamik siteler, gelen HTTP isteğine göre her kullanıcıya özel içerik üretir.

İstemci tarafı

Tarayıcıda çalışır (HTML, CSS, JavaScript). Arayüzü ve kullanıcı etkileşimini yönetir.

Sunucu tarafı

Web sunucusunda çalışır. Veritabanıyla konuşur, kullanıcıyı doğrular, hazır HTML üretir.

Neden kullanışlı?

Veri tek yerde saklanır, içerik kullanıcıya göre değişir, kritik işlemler kullanıcıdan gizli kalır.

Örnek

Kütüphane sitesinde kitap listesi ve üyenin ödünç aldıkları veritabanından çekilip sayfaya yazılır.

İstemci-Sunucu: Bir İstek Nasıl İşlenir?

Fark anlaşıldı; şimdi tarayıcı ile sunucu arasında bir isteğin yolculuğuna bakalım.

1. İstek	Kullanıcı bir bağlantıya tıklar; tarayıcı sunucuya HTTP isteği (adres, yöntem, veri) gönderir.
2. Yönlendirme	Sunucu kodu adrese bakıp hangi kodun çalışacağına karar verir.
3. İşleme	Gerekirse veritabanından veri okunur ya da yazılır, kullanıcının izinleri kontrol edilir.
4. Şablon	Veriler bir HTML şablonuna yerleştirilerek sayfa oluşturulur.
5. Yanıt	Sunucu sayfayı HTTP yanıtı olarak gönderir; tarayıcı onu gösterir.

Web Çerçeveleri ve Güvenlik

Bu akıştaki işleri tek tek yazmak yerine çerçeveler kullanılır.

Çerçeve neden gerekli?

Adres yönlendirme, veritabanı erişimi, şablon, oturum ve form gibi sık işleri hazır sunar; her şeyi sıfırdan yazmaktan kurtarır.

Doğru çerçeveyi seçmek

İlk sunucu tarafı uygulaman için çerçeve seçmek önemlidir. Bu derste ikisine bakıyoruz: Django (Python) ve Express (Node.js).

Güvenlik her aşamada

Sitenin kurulumunda ve işletilmesinin her aşamasında dikkat gerekir.

Kullanıcı verisini korumak

Çerçeveler yaygın tehditlere karşı yerleşik korumalar sunar; Django bölümünde örneklerini göreceğiz.

Django (Python): Tanıtım ve Kurulum

İki çerçeveden ilki: Python ile yazılmış Django.

Django nedir?

Python ile yazılmış, son derece popüler ve tam özellikli bir çerçeve. Admin sitesi, oturum, kimlik doğrulama, form ve test desteği hazır gelir.

Kurulum

Windows, Linux (Ubuntu) ve macOS'ta geliştirme ortamı kurulur ve test edilir; ardından LocalLibrary eğitimi başlar.

AÇIKLAMA

Django, "bataryalar dahil" yaklaşımıyla çalışan bir çerçevedir: kimlik doğrulama, yönetici arayüzü, oturum, form ve test araçları ayrı kütüphane aranmadan hazır gelir. Bu yüzden büyük siteleri hızlı kurmak isteyenler için uygundur. Derste Django'yu LocalLibrary örnek projesiyle adım adım öğreneceğiz.

Django'da Dört Temel Parça

İstek akışındaki adımların Django karşılığı; eğitimdeki tüm sayfalar bunlarla yazılır.

Model	Verinin yapısını tanımlar; Django veriyi veritabanında saklar ve değiştirir.
URL eşleme	Gelen adresi ilgili görünüme bağlar.
Görünüm	İsteği işler, modellerden veriyi alır.
Şablon	Veriyi HTML sayfasına dönüştürür.

AÇIKLAMA

Kullanıcı bir adrese girdiğinde önce URL eşleme devreye girer ve isteği doğru görünüme gönderir. Görünüm gerekirse modeller aracılığıyla veritabanından veri alır, sonra bu veriyi bir şablona verir. Şablon HTML sayfasını üretir ve yanıt olarak kullanıcıya döner. Bu dört adımı aklında tut; sonraki tüm örnekler bunun tekrarıdır.

Django Eğitimi 1–4: Proje Temeli

Bu parçalarla LocalLibrary sitesinin temelini kuruyoruz.

1. Yerel kütüphane sitesi	Örnek proje LocalLibrary: kitapları, yazarları ve ödünç alınan kitapları yöneten site.
2. İskelet site	Ayarlar, URL'ler, modeller, görünümler ve şablonlar için doldurulacak iskelet proje.
3. Modeller	Veri yapıları ve alan türleri tanımlanır; veritabanı işini Django yapar.
4. Yönetici sitesi	Modeller admin sitesine kaydedilir; giriş yapılıp gerçek kitap verisi eklenir.

Kod Örneği: Django Modeli

Model, bir Python sınıfıdır; her sınıf bir veritabanı tablosuna karşılık gelir.

models.py

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(
        'Author', on_delete=models.SET_NULL, null=True)
    summary = models.TextField(max_length=1000)

    def __str__(self):
        return self.title
```

Alan türleri

CharField kısa metin, TextField uzun metin, ForeignKey başka bir modele bağlantıdır.

Veritabanı işi

Tabloyu elle yazmayız; Django modelden veritabanı yapısını üretir.

__str__

Admin sitesinde kaydın okunur adını belirler.

Django Eğitimi 5–8: Sayfalar ve Kullanıcılar

Temel hazır; şimdi siteye sayfalar ve kullanıcı özellikleri ekliyoruz.

5. Ana sayfa	Her model türünden kaç kayıt olduğunu ve yan çubuk bağlantılarını gösterir.
6. Liste ve detay	Kitap ve yazar sayfaları genel sınıf tabanlı görünümlerle az kodla yazılır.
7. Oturumlar	Oturum tabanlı ziyaret sayacı; anonim kullanıcılar için kalıcı davranış sağlar.
8. Kimlik doğrulama	Giriş-çıkış ve izinlere göre erişim; kullanıcıya ve personele özel sayfalar.

Kod Örneği: URL, Görünüm ve Şablon

Ana sayfa için dört parçanın birlikte çalışması:

urls.py ve views.py

```
# urls.py
urlpatterns = [
    path('', views.index, name='index'),
]

# views.py
def index(request):
    num_books = Book.objects.count()
    context = {'num_books': num_books}
    return render(request, 'index.html', context)
```

URL eşleme

Boş adres (ana sayfa) index görünümüne bağlanır.

Görünüm

Book modelinden kayıt sayısını alır ve context sözlüğüne koyar.

render

Veriyi index.html şablonuna gönderir; şablonda {{ num_books }} olarak yazdırılır.

Django Eğitimi 9–10: Formlar ve Test

Kullanıcıdan veri almak ve değişikliklerin siteyi bozmadığından emin olmak.

9. Formlarla çalışma

HTML formlarıyla model kayıtları oluşturulur, güncellenir ve silinir. Kütüphaneciler admin yerine kendi formlarını kullanır.

10. Uygulamayı test etme

Site büyüdükçe elle test zorlaşır. Otomatik birim testleri her değişiklikten sonra kolayca ve güvenilir çalışır; Django'nun test çerçevesi kullanılır.

AÇIKLAMA

Formlar kullanıcıdan veri almanın standart yoludur. Django'da form kullanarak admin sitesine ihtiyaç duymadan kayıt eklenir, düzenlenir ve silinir. Testler ise güvence verir: bir değişiklik başka bir yeri bozduysa otomatik testler bunu hemen gösterir. Site büyüdükçe elle test etmek sürdürülebilir olmaktan çıkar.

Kod Örneği: Django Testi

Test, kodun beklediğimiz sonucu verdiğini otomatik olarak kontrol eder.

tests.py

```
from django.test import TestCase
from .models import Book

class BookModelTest(TestCase):
    def test_str_returns_title(self):
        book = Book(title='Sefiller')
        self.assertEqual(str(book), 'Sefiller')
```

TestCase

Her test, ayrı bir test veritabanıyla çalışır; gerçek veriyi etkilemez.

assertEqual

Beklenen değerle gerçek değeri karşılaştırır; farklıysa test başarısız olur.

Çalıştırma

python manage.py test komutu tüm testleri çalıştırır.

Django: Dağıtım ve Güvenlik

Site hazır ve test edildi; sıra internete açmakta ve korumakta.

11. Üretime dağıtma

Site halka açık bir sunucuya kurulur. Barındırma hizmeti seçilir ve site üretime hazırlanır.

Django web uygulaması güvenliği

Web güvenliği makalesindeki yaygın tehditlere karşı Django'nun yerleşik korumalarının nasıl çalıştığı gösterilir.

AÇIKLAMA

Dağıtım, sitenin geliştirme bilgisayarından çıkıp gerçek kullanıcıların erişebildiği bir sunucuya taşınmasıdır. Bunun için bir barındırma hizmeti seçilir ve site üretim ortamına uygun hale getirilir. Güvenlik tarafında Django, yaygın saldırılara karşı yerleşik önlemler sunar; ancak bunları doğru kullanmak geliştiricinin sorumluluğundadır.

Express (Node.js): Tanıtım ve Kurulum

İkinci çerçeve: aynı fikir, bu kez JavaScript ile.

Node ve Express nedir?

Node.js, JavaScript'in sunucuda çalışmasını sağlayan ortamdır. Express, onun üzerinde çalışan popüler web çerçevesidir.

Kurulum

Windows, Linux (Ubuntu) ve macOS'ta Node/Express geliştirme ortamı kurulur ve test edilir.

AÇIKLAMA

Node.js sayesinde tarayıcıdan bildiğimiz JavaScript sunucuda da kullanılır; tek dille hem arayüz hem sunucu yazılabilir. Express bu ortamın üzerine rota, istek ve yanıt yönetimi gibi temel web işlerini kolaylaştıran bir katman ekler. Eğitimde Django'daki aynı LocalLibrary projesini Express ile yapacağız.

Express Uygulamasının Yapı Taşları

Django'daki dört parçanın Express karşılığı:

Rota	Gelen adresi ilgili işleyiciye yönlendirir (URL işleme kodu).
Denetleyici	İsteği işleyen fonksiyon; veriyi alır ve yanıtı hazırlar.
Model (Mongoose)	Şema ve modellerle veritabanı erişimi, alan türleri ve doğrulama.
Şablon (Pug)	Veriyi HTML sayfasına dönüştürür.

AÇIKLAMA

Bir istek geldiğinde rota adresi eşleştirir ve ilgili denetleyici fonksiyonunu çağırır. Denetleyici, Mongoose modeli üzerinden veritabanından veri alır ve bu veriyi Pug şablonuna verir. Şablon HTML'i üretir ve yanıt olarak gönderilir. Django'daki URL eşleme, görünüm, model ve şablon dörtlüsünün Express'teki karşılığı budur.

Express Eğitimi 1–4: Proje Temeli

Aynı LocalLibrary projesini bu kez Express ile kuruyoruz.

1. Yerel Kütüphane sitesi	Aynı LocalLibrary örnek projesi; neler öğrenileceği anlatılır.
2. İskelet site	Rotalar, şablonlar/görünümler ve veritabanları için iskelet proje.
3. Mongoose ile veritabanı	Veritabanına giriş; şemalar, modeller, alan türleri ve temel doğrulama.
4. Rotalar ve denetleyiciler	Her uç nokta için 'sahte' işleyicilerle modüler rota yapısı kurulur; sonra gerçekleriyle doldurulur.

Kod Örneği: Mongoose Şeması

Express tarafında veri yapısını şema ve model ile tanımlıyoruz.

models/book.js

```
const mongoose = require("mongoose");
const Schema = mongoose.Schema;

const BookSchema = new Schema({
  title: { type: String, required: true },
  summary: { type: String, required: true },
  author: { type: Schema.Types.ObjectId, ref: "Author" },
});

module.exports = mongoose.model("Book", BookSchema);
```

Şema

Belgenin alanlarını ve türlerini tanımlar; Django'daki model alanlarının karşılığıdır.

required

Doğrulama kuralı: bu alan boş bırakılamaz.

ref

Başka bir modele (Author) bağlantı kurar; model dışa aktarılıp rotalarda kullanılır.

Kod Örneği: Rota ve Denetleyici

Rota adresi seçer, denetleyici işi yapar:

routes ve controllers

```
// routes/catalog.js
router.get('/books', book_controller.book_list);

// controllers/bookController.js
exports.book_list = async (req, res) => {
  const books = await Book.find().sort({ title: 1 });
  res.render('book_list', { title: 'Kitaplar', books });
};
```

Rota

GET isteğiyle gelen /books adresini bir denetleyici fonksiyonuna bağlar.

Denetleyici

Mongoose modelinden veriyi alır; async/await ile sonucu bekler.

res.render

Veriyi Pug şablonuna gönderir; oluşan HTML yanıt olarak döner.

Express Eğitimi 5–7: Sayfalar, Formlar, Dağıtım

Temel hazır; sayfalar, formlar ve dağıtımla site tamamlanıyor.

5. Verilerin görüntülenmesi	Kayıt sayılarını gösteren ana sayfa ve her model için liste ile detay sayfaları.
6. Formlarla çalışma	Pug ile HTML formları; veritabanında belge oluşturma, güncelleme ve silme.
7. Üretime dağıtım	Halka açık sunucuya kurulum: barındırma hizmeti bulma ve üretime hazırlık.

AÇIKLAMA

Bu bölümde sitenin geri kalanı tamamlanır. Önce veritabanındaki kitap ve yazar bilgileri liste ve detay sayfalarında gösterilir. Ardından formlar eklenerek kullanıcıların kayıt oluşturması, güncellemesi ve silmesi sağlanır. Son olarak site halka açık bir sunucuya yüklenerek herkesin erişimine açılır.

Ek Eğitimler

Çerçeve dışında, sunucu tarafında işe yarayan üç konu.

Apache .htaccess	Ana yapılandırma dosyasını değiştirmeden web sunucusu dizinlerini yapılandırmayı sağlar.
Sunucu MIME türleri	İçeriğin medya türünü tanımlar; tarayıcıya içeriğin nasıl işleneceği konusunda ipucu verir.
Basit Node.js sunucusu	Framework kullanmadan, yerleşik API'lerle birkaç satırda statik dosya sunucusu yazılır.

AÇIKLAMA

Bu üç konu bir çerçeveye bağlı değildir; hangi dille çalışırsan çalış sunucu tarafında karşına çıkar. .htaccess, Apache sunucusunda klasör bazında ayar yapmanı sağlar. MIME türü yanlışsa tarayıcı dosyayı yanlış yorumlayabilir. Basit Node.js sunucusu ise bir isteği alıp dosya göndermeyi çerçevesiz, çıplak haliyle gösterir.

Kod Örneği: Basit Node.js Sunucusu

Framework olmadan, yerleşik modüllerle çalışan en küçük sunucu fikri:

server.js

```
const http = require("http");
const fs = require("fs");

http.createServer((req, res) => {
  fs.readFile("index.html", (err, data) => {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end(data);
  });
}).listen(8000);
```

createServer

Her gelen istek için çalışan bir fonksiyon verilir.

Content-Type

MIME türüdür; tarayıcı içeriği buna göre işler.

Yerleşik modüller

http ve fs ile çerçeve olmadan dosya sunulabilir.

Django ve Express

Aynı proje, iki çerçeve: ortak fikir ve farklar.

	Django	Express
Dil	Python	JavaScript (Node.js)
Yapı	Tam özellikli çerçeve	Node.js üzerinde popüler çerçeve
Veri katmanı	Django modelleri	Mongoose şema ve modelleri
Adres yönlendirme	URL eşleme	Rotalar ve denetleyiciler
Şablon ve form	Django şablonları ve formları	Pug ile şablon ve formlar
Eğitim	11 bölüm	7 bölüm
Ortak proje	LocalLibrary	LocalLibrary

Neler Öğrendik?

Başa dönüp hedeflerle karşılaştıralım:

Sunucu tarafı	Kod sunucuda çalışır; istek gelir, veri işlenir, HTML yanıt döner.
Django	Model, URL eşleme, görünüm ve şablon; admin, oturum, form ve test hazır gelir.
Express	Rota, denetleyici, Mongoose modeli ve Pug şablonu; JavaScript ile aynı fikir.
Ortak noktalar	İkisinde de aynı proje yapılır; test, güvenlik ve dağıtım adımları benzerdir.

Kaynaklar

Bu sunumun dayandığı materyaller:

MDN Web Docs	developer.mozilla.org: Sunucu tarafı web sitesi programlama modülü (Django ve Express eğitimleri).
Ders materyali	SCÜ Şarkışla UBYO Web Programlama I dersi için çevrilmiş MDN içerikleri.
Kod örnekleri	LocalLibrary örneğinden esinlenilmiş, sadeleştirilmiş örnekler; tam kodlar için MDN eğitimlerine bakın.

MDN WEB DOCS TEMELLİ DERS

Web Performansı

Hızlı sayfa, mutlu kullanıcı

Web performansı neden önemli?

NEDİR?

Performans, kullanıcıyı sitede tutan ve işi büyüten temel bir kalite ölçütüdür. Yavaş sayfa yalnızca can sıkmaz; erişilebilirliği, maliyeti ve enerji tüketimini de etkiler. Yavaş bağlantıdaki ve eski cihazdaki kullanıcı en çok zarar görendir.

NASIL ÇALIŞIR?

Sayfa geç açılınca kullanıcı vazgeçer; bu da dönüşüm oranını ve geliri düşürür. Büyük dosyalar mobil veri kotasını harcar ve cihazın pilini daha hızlı bitirir.

NEDEN ÖNEMLİ?

Hız; kullanıcı memnuniyetinin, erişilebilirliğin ve arama görünürlüğünün temelidir.

ÖZETLE

Performans bir lüks değil, kullanıcının hakkıdır.

veri-tasarrufu.js

```
// Veri tasarrufu açıksa hafif sürüm
const c = navigator.connection;
if (c && c.saveData) {
  yukleHafifSurum();
} else {
  yukleTamSurum();
}
```

Not: navigator.connection her tarayıcıda yoktur; önce varlığını kontrol edin.

Günlük hayatta: Kapısı hep kilitli ya da kuyruğu çok uzun bir dükkâna müşteri ikinci kez gelmez.

Web performansı nedir?

NEDİR?

Web performansı, bir sayfanın ne kadar hızlı yüklendiğini ve ne kadar akıcı hissettirdiğini anlatır. İki yönü vardır: gerçek performans milisaniyeyle ölçülür, algılanan performans ise kullanıcının hissidir. İyi bir deneyim için ikisi de gerekir.

NASIL ÇALIŞIR?

FCP ilk içeriğin, LCP en büyük içeriğin görünme süresidir. CLS ani yer değiştirmeyi, INP ise tıklamaya yanıt gecikmesini ölçer.

NEDEN ÖNEMLİ?

Yavaş sayfada kullanıcı siteyi terk eder, satışlar düşer ve arama sıralaması olumsuz etkilenir.

ÖZETLE

Hızlı sayfa, mutlu kullanıcı demektir. Önce ölç, sonra optimize et.

LCP ölçümü

```
1  // LCP ölçümü
2  new PerformanceObserver((list) => {
3    const e = list.getEntries().at(-1);
4    console.log("LCP:", e.startTime);
5  }).observe({
6     type: "largest-contentful-paint",
7     buffered: true });
```

Not: Ölçmeden optimize etmeyin; önce gerçek kullanıcı verisini toplayın.

Günlük hayatta: Restoranda yemek lezzetli olsa da 40 dakika beklemek, müşteriyi kaçırır.

Kullanıcı performansı nasıl algılar?

NEDİR?

Algılanan performans, sayfanın gerçekte ne kadar hızlı yüklendiğinden çok kullanıcının onu ne kadar hızlı hissettiğidir. Aynı süre, bekleme sırasında ekranda bir şey görünüyorsa daha kısa algılanır.

NASIL ÇALIŞIR?

İskelet ekranlar, yer tutucular, ilerleme göstergeleri ve önce görünür alanın yüklenmesi bekleme hissini azaltır. Tıklamaya hemen tepki veren arayüz, ağır ama donuk bir arayüzden hızlı hissettirir.

NEDEN ÖNEMLİ?

Kullanıcı kronometre tutmaz; bekleme sırasında ne gördüğüne ve arayüzün tepki verip vermediğine göre karar verir.

ÖZETLE

Boş ekran bırakma; kullanıcıya hemen bir şey göster.

iskelet.css

```css
/* Yüklenirken gösterilen yer tutucu */
.iskelet {
    background: lightgray;
    border-radius: 8px;
    height: 120px;
    animation: nabiz 1.2s infinite;
}
```

Not: Yer tutucunun boyutu gerçek içerikle aynı olursa sayfa zıplamaz.

Günlük hayatta: Doktor kapısındaki "sıranız yaklaşıyor" ekranı, boş bir bekleme salonundan daha kısa hissettirir.

Kritik render yolu

NEDİR?

Kritik render yolu, tarayıcının HTML, CSS ve JavaScript'i alıp ekranda piksellere dönüştürdüğü adımlar zinciridir. Kullanıcı sayfada ilk görüntüyü bu zincir bitince görür; bu yüzden zincirdeki her gereksiz bekleme, ekranın boş kalma süresini uzatır.

NASIL ÇALIŞIR?

HTML'den DOM, CSS'ten CSSOM oluşur; ikisi birleşip render ağacını verir. Sonra yerleşim (layout) ve boyama (paint) yapılır. CSS ve senkron JS bu zinciri bekletebilir.

NEDEN ÖNEMLİ?

Zincir ne kadar kısa ve hafifse ilk görüntü o kadar erken gelir. En kolay kazanç, gereksiz CSS'i media ile koşullu yüklemektir.

ÖZETLE

İlk ekran için gerekeni önce, gerisini sonra yükle.

Koşullu CSS

```
1  <!-- ekranı engeller -->
2  <link rel="stylesheet" href="main.css">
3  <!-- yalnızca yazdırırken -->
4  <link rel="stylesheet" href="print.css"
5    media="print">
```

Not: media="print" olan dosya, ekran render'ını engellemez.

Günlük hayatta: Ev teslim edilmeden boya bitmelidir; kullanılmayan odanın boyasını beklemek vakit kaybıdır.

Hedefler ve performans bütçeleri

NEDİR?

Gecikme (latency), isteğin gidip cevabın gelmesine kadar geçen bekleme süresidir. "Hızlı" olmanın ne demek olduğunu somut hedeflerle, bu hedefleri korumayı ise performans bütçesiyle belirlersiniz.

NASIL ÇALIŞIR?

Yaklaşık hedefler: etkileşime yanıt 100 ms, animasyon karesi 16 ms, sayfa içeriği 1 saniye. Bütçe ise bir üst sınırdır (örneğin script en fazla 170 KB); Lighthouse gibi araçlar her derlemede kontrol eder.

NEDEN ÖNEMLİ?

Hedef ve sınır olmadan sayfa zamanla şişer, iyileştirmenin ne zaman bittiği de belli olmaz.

ÖZETLE

Önce hedefi yaz, sonra her değişiklikte kontrol et.

butce.js

```
// Hedefler (ms) ve bütçe (KB)
const hedef = { yanit: 100, kare: 16 };
const butce = {
   script: 170,
   gorsel: 500
};
```

Not: Bütçeyi aşan değişikliği yayına almadan önce durdurun.

Günlük hayatta: Yola çıkmadan "en geç 30 dakikada varmalıyım" demek ve aylık harcama limiti koymak gibi.

Ölçüm araçları ve API'leri

NEDİR?

Ölçmeden yapılan optimizasyon tahmindir. Ölçüm, sayfanın hangi aşamada ne kadar zaman harcadığını gösterir ve değişikliğin işe yarayıp yaramadığını kanıtlar.

NASIL ÇALIŞIR?

Genel rapor için PageSpeed Insights ve WebPageTest, kayıt için DevTools Performans paneli kullanılır. Kodla ölçmek için Navigation, Resource, User ve Server Timing API'leri ile PerformanceObserver vardır.

NEDEN ÖNEMLİ?

Darboğaz çoğu zaman tahmin ettiğimiz yerde değildir; gerçek kullanıcı verisi sorunu gösterir.

ÖZETLE

Ölç, değiştir, tekrar ölç.

olcum.js

```
// Ana belge ve kendi işaretin
const n = performance
  .getEntriesByType("navigation")[0];
console.log("TTFB:", n.responseStart);
performance.mark("basla");
performance.measure("ekran", "basla");
```

Not: Laboratuvar ölçümünü gerçek kullanıcı verisiyle birlikte değerlendirin.

Günlük hayatta: Kiloyu tahminle değil tartıyla takip edersiniz; değişimi ancak ölçerek görürsünüz.

Medya: görsel, video ve tembel yükleme

NEDİR?

Görseller ve videolar sayfanın en ağır kalemleridir. Biçim, boyut ve ne zaman yükleneceği performansı belirler; tembel yükleme ise ekran dışındakini gerektiğinde indirir.

NASIL ÇALIŞIR?

Görselde srcset, sizes, WebP/AVIF ve width/height kullanın. Videoda WebM ve MP4 sunun, preload ile sınırlayın, poster verin. Ekran dışı görsel ve iframe için loading="lazy" ekleyin.

NEDEN ÖNEMLİ?

Daha az bayt, daha az bekleme ve daha az içerik kayması (CLS) demektir.

ÖZETLE

Doğru boyut, doğru biçim, doğru zaman. İlk ekran görselini lazy yapma.

medya.html

```
<img src="foto.webp" loading="lazy"
  srcset="kucuk.webp 480w,
  buyuk.webp 1200w"
  width="1200" height="800"
alt="Manzara">
<video controls preload="none"
  poster="kapak.webp" width="640">
  <source src="tanitim.webm"
    type="video/webm">
</video>
```

Not: Sayfanın en büyük (LCP) görselini lazy yapmayın.

Günlük hayatta: Arkadaşa 20 MB'lık orijinali değil küçük halini gönderir, misafire yemeği istedikçe servis edersiniz.

Kaynak yükleme stratejileri

NEDİR?

Tarayıcıya hangi kaynağı ne zaman indireceğini biz söyleyebiliriz. Script, font ve stil dosyaları ilk görüntüyü geciktirebilir; doğru sıra ve önceden hazırlık sayfayı hızlandırır.

NASIL ÇALIŞIR?

defer, script'i HTML'i bekletmeden indirip sırayla çalıştırır; async sıra gözetmez. preload kritik kaynağı erkenden indirir, preconnect ve dns-prefetch bağlantıyı önceden hazırlar, Speculation Rules sonraki sayfayı önceden yükler.

NEDEN ÖNEMLİ?

Bekleme sürelerinin çoğu, kaynak istenmeden önce halledilerek ortadan kalkar.

ÖZETLE

Kritik olanı öne al, bildiğin bağlantıyı hazırla, gerisini ertele.

yukleme.html

```
<script src="app.js" defer></script>
<link rel="preload" href="font.woff2"
  as="font" type="font/woff2"
crossorigin>
<link rel="preconnect"
  href="https://cdn.ornek.com">
<link rel="prefetch" href="sonraki.js">
```

Not: Her şeyi preload etmek ters teper; yalnızca kritik kaynağı işaretleyin.

Günlük hayatta: Yemek pişerken malzemeyi önceden hazırlar, misafir gelmeden masayı kurarsınız.

CSS ve fontlar

NEDİR?

CSS render'ı engeller: stil dosyası inmeden tarayıcı sayfayı boyamaz. Web fontları ise ek dosyadır ve metnin görünmesini geciktirebilir.

NASIL ÇALIŞIR?

CSS'i küçültün, kullanılmayanı temizleyin, ilk ekran stilini ayırın, content-visibility ile ekran dışını erteleyin. Fontta WOFF2, alt küme (subset), font-display ve kritik fontu preload kullanın.

NEDEN ÖNEMLİ?

Hafif CSS erken ilk boyama sağlar; doğru font yüklemesi LCP ve CLS'i korur.

ÖZETLE

Küçük CSS, az font, metin hemen görünsün.

stil.css

```
@font-face {
    font-family: "Okul";
    src: url("okul.woff2")
format("woff2");
    font-display: swap;
}
.alt-bolum {
    content-visibility: auto;
}
```

Not: content-visibility için tarayıcı desteğini MDN'den kontrol edin.

Günlük hayatta: Valize yalnızca lazım olanı koyarsınız; özel yazı tipi yoksa bilgisayar yedeğini kullanır, metin okunur.

JavaScript ve başlangıç performansı

NEDİR?

JavaScript ana iş parçacığında çalışır; çizim, tıklama yanıtı ve kod aynı kuyruğu paylaşır. Başlangıç performansı ise uygulamanın açıldığı andan kullanıma hazır olana kadar geçen süredir.

NASIL ÇALIŞIR?

50 ms'den uzun görevler (long task) kuyruğu tıkar. Ağır işi Web Worker'a taşıyın, kodu bölüp (code splitting) gerektiğinde yükleyin, açılışta yalnızca gerekeni çalıştırın.

NEDEN ÖNEMLİ?

Kuyruk tıkanınca sayfa donar ve INP kötüleşir; yavaş açılışta kullanıcı ayrılır.

ÖZETLE

Ana iş parçacığını meşgul etme, açılışta sadece gerekeni yap.

js.js

```
// Ağır işi arka plana al
const worker = new Worker("hesap.js");
worker.postMessage(veri);
// Açılış dışı işi ertele
requestIdleCallback(() => {
  import("./analitik.js");
});
```

Not: Worker DOM'a erişemez; requestIdleCallback bazı tarayıcılarda yoktur.

Günlük hayatta: Ağır işi ikinci kasaya verirsiniz; dükkânı açarken önce kapıyı ve kasayı hazırlarsınız.

Animasyon performansı

NEDİR?

Akıcı animasyon, her karenin yaklaşık 16 ms içinde çizilmesini gerektirir (saniyede 60 kare). Bir kare geç kalırsa kullanıcı takılma hisseder; hangi CSS özelliğini canlandırdığınız bu süreyi belirler.

NASIL ÇALIŞIR?

transform ve opacity, yerleşim hesabı gerektirmeden çoğunlukla GPU ile çizilir. width, top ve left gibi özellikler ise her karede yeniden yerleşim tetikler.

NEDEN ÖNEMLİ?

Her karede yerleşimi yeniden hesaplamak ana iş parçacığını yorar ve animasyonu takılgan yapar.

ÖZETLE

Animasyonda transform ve opacity kullan.

animasyon.css

```
/* Akıcı: yerleşimi bozmaz */
.kutu {
    transition: transform 0.3s;
}
.kutu:hover {
    transform: translateX(40px);
}
```

Not: Takılma görürseniz Performans panelinde kare hızını inceleyin.

Günlük hayatta: Mobilyayı yerden sürüklemek yerine tekerlekli tablaya koyup kaydırmak; aynı hareket, çok daha az efor.

Mobil performans

NEDİR?

Mobil cihazlarda bant genişliği, işlemci gücü ve pil sınırlıdır. Masaüstünde hızlı görünen bir sayfa telefonda yavaş ve pil tüketen bir deneyime dönüşebilir.

NASIL ÇALIŞIR?

Mobil için daha küçük görseller ve daha az JavaScript sunun, test ederken zayıf cihazı ve yavaş ağı simüle edin, dokunmaya anında yanıt veren bir arayüz hedefleyin.

NEDEN ÖNEMLİ?

Web kullanımının büyük bölümü telefondan yapılır; ortalama cihaz geliştirici bilgisayarından çok daha zayıftır.

ÖZETLE

Önce telefonu ve yavaş ağı düşünerek tasarla.

mobil.html

```
<meta name="viewport"
  content="width=device-width,
  initial-scale=1">
<img srcset="mobil.webp 480w,
  masaustu.webp 1200w"
  sizes="100vw" alt="Kapak">
```

Not: DevTools'ta cihaz ve ağ kısıtlama (throttling) ile test edin.

Günlük hayatta: Dar sokakta kamyonla değil küçük araçla gidersiniz; yol aynı, ilerleme çok daha rahat.

Önbellek ve sıkıştırma

NEDİR?

Önbellek, bir kez indirilen dosyaları tarayıcıda veya aradaki sunucularda saklayıp tekrar kullanmaktır. Sıkıştırma ise sunucudan gönderilen veriyi küçültür. İkisi birlikte hem indirilecek veri miktarını hem de bekleme sayısını azaltır.

NASIL ÇALIŞIR?

Cache-Control, dosyanın ne kadar saklanacağını söyler. Dosya adına sürüm özeti (hash) eklenirse uzun süre güvenle saklanır. Brotli veya gzip metni küçültür; CDN içeriği yakından sunar.

NEDEN ÖNEMLİ?

Daha az veri daha az bekleme demektir; tekrar ziyaretlerde sayfa neredeyse anında açılır.

ÖZETLE

Bir kez indir, çok kez kullan; gönderirken küçült.

HTTP başlıkları

```
1  # Yanıt başlıkları
2  Cache-Control: public,
3    max-age=31536000, immutable
4  Content-Encoding: br
```

Not: Brotli (br) genellikle gzip'ten daha iyi sıkıştırır.

Günlük hayatta: Sık aradığınız numarayı rehbere kaydedersiniz; her seferinde yeniden aramazsınız.

Önce / Sonra: Bir sayfayı hızlandırmak

ÖNCE · Yavaş sürüm

index.html

```
<!-- her şey engelleyici -->
<script src="app.js"></script>
<link rel="stylesheet" href="all.css">
<img src="foto.png" alt="Kapak">
<img src="galeri.png" alt="Galeri">
```

LCP	CLS	Boyut
4,8 sn	0,35	3,2 MB

SONRA · Hızlı sürüm

index.html

```
<script src="app.js" defer></script>
<link rel="stylesheet" href="all.css">
<img src="foto.webp" width="1200"
  height="800" fetchpriority="high"
alt="Kapak">
<img src="galeri.webp" loading="lazy"
  width="600" height="400" alt="Galeri">
```

LCP	CLS	Boyut
1,9 sn	0,02	0,9 MB

Değişenler: defer, WebP, width/height, lazy, fetchpriority. Değerler örnek senaryodur.

Günlük hayatta: Aynı yolu, trafiksiz saatte ve doğru şeritten gitmek gibi; varış aynı, süre çok kısa.

Özet ve kaynaklar

NEDİR?

Web performansı tek bir ayar değil; ölçmekten görsele, koddan ağa kadar uzanan bir alışkanlıktır. MDN'nin performans bölümü, bu konuların her biri için ayrıntılı rehber ve API belgeleri sunar.

NASIL ÇALIŞIR?

Çalışırken şu sırayı izleyin: hedef koy, ölç, en büyük darboğazı bul, düzelt, tekrar ölç. Kaynak olarak MDN'deki Web Performance bölümünü, Learn modülünü ve Performance API belgelerini kullanın.

NEDEN ÖNEMLİ?

Araçlar ve ölçütler zamanla değişir; resmi belgeyi takip etmek bilgiyi güncel tutar.

ÖZETLE

Hedef, ölçüm, düzeltme, tekrar.

kontrol-listesi.md

```
// Performans kontrol listesi
// 1. Hedef belirle
// 2. Ölç (Lighthouse, DevTools)
// 3. Darboğazı bul
// 4. Düzelt (görsel, JS, CSS, font)
// 5. Tekrar ölç
```

Not: Kaynak: developer.mozilla.org, Web performance bölümü.

Günlük hayatta: Spor gibi; bir kez yapmak yetmez, düzenli ölçüp ayarlayarak ilerlersiniz.

MDN WEB DOCS · LEARN WEB DEVELOPMENT

Web Testi

Her tarayıcıda ve her cihazda çalışan web siteleri

DERS

Dersin konuları

1 · Giriş

Tarayıcılar arası uyumluluk testi ve test süreci

2 · Test stratejileri

Hangi tarayıcı ve cihazların, nasıl test edileceği

3 · HTML ve CSS

Sık görülen sorunlar ve çözüm yolları

4 · Özellik tespiti

Desteği kontrol edip alternatif sunmak

5 · Otomasyon

Gulp, ticari servisler ve Selenium

Önkoşul: HTML, CSS ve JavaScript temelleri. Kaynak: MDN › Learn web development › Testing

1 · GİRİŞ

Tarayıcılar arası uyumluluk testi nedir?

Web sitesinin farklı tarayıcı ve cihazlarda doğru çalıştığını kontrol etme işidir (cross-browser testing). Geliştirici şunları göz önünde bulundurmalıdır:

Tarayıcılar

Yeni JavaScript ve CSS özelliklerini desteklemeyen eski sürümler de dahil.

Cihazlar

Masaüstü ve dizüstü bilgisayarlar, tabletler, telefonlar ve akıllı televizyonlar; donanım özellikleri birbirinden çok farklıdır.

Engelli kullanıcılar

Ekran okuyucu gibi yardımcı teknolojileri kullanan ya da yalnızca klavyeyle gezinen kişiler.

Kendi cihazında sorunsuz çalışan bir site, her kullanıcıda çalışacak demek değildir.

Her tarayıcıda aynı görünüm gerekmez; temel işlevlerin her yerde kullanılabilmesi yeterlidir. Hangi tarayıcı ve cihazların destekleneceği, site sahibiyle önceden kararlaştırılır.

1 · GİRİŞ

Uyumluluk sorunlarının nedenleri ve test süreci

Tarayıcı hataları ve farklılıkları

Tarayıcılarda hata olabilir ya da bir özellik farklı uygulanmış olabilir. 1990'larda üreticiler bunu rekabet için bilerek yapıyordu; bugün standartlara uyum çok daha iyi, ama farklar tamamen kalkmadı.

Farklı destek seviyeleri

Yeni özellikler her tarayıcıda yoktur. Geliştirilmesi bırakılmış eski tarayıcılar, yeni özellikler çıkmadan önce dondurulmuş olabilir. Gerekirse kod eski söz dizimine dönüştürülür.

Cihaz sınırlamaları

Masaüstü için tasarlanan bir site telefonda küçük ve okunaksız görünebilir. Ağır animasyonlar düşük donanımlı cihazlarda yavaş çalışır.

Test süreci dört aşamadan oluşur (2–4. aşamalar gerektiği kadar tekrarlanır)

1 · Planlama

Hedef kitlenin kullandığı tarayıcı ve cihazları belirle; sorun çıkabilecek alanları listele.

→

2 · Geliştirme

İşi modüllere böl ve her parçayı eklemeden önce test et.

→

3 · Test ve keşif

Önce kararlı tarayıcılarda, sonra hedef listedeki tüm tarayıcılarda dene.

→

4 · Düzeltme

Hatayı daralt, nedenini bul, düzelt ve testi yeniden yap.

Önce kendi kodundaki hataları gider. Testi projenin sonuna bırakmak, hataların düzeltilmesini çok daha pahalı ve zahmetli hale getirir.

Geliştirme yaklaşımları ve test adımları

Üç geliştirme yaklaşımı

• Her hedef tarayıcıda işlevi mümkün olduğunca aynı çalıştırmak (farklı kod yolları, polyfill ya da kütüphane kullanarak).

• Bazı özelliklerin her yerde aynı çalışmayacağını kabul edip desteklemeyen tarayıcılara kabul edilebilir bir alternatif sunmak.

• Bazı eski tarayıcılarda sitenin çalışmayacağını kabul etmek (site sahibi ve kullanıcılar bunu kabul ediyorsa).

Test sırası

• Önce iki üç kararlı tarayıcıda dene (Firefox, Chrome, Safari, Edge).

• Basit bir erişilebilirlik testi yap: siteyi yalnızca klavye ve ekran okuyucuyla kullanmayı dene.

• Android ya da iOS gibi bir mobil platformda dene.

• Ardından hedef kitlenin kullandığı tüm tarayıcılara geç.

Hata bulunduğunda: hangi platform, cihaz ve sürümde görüldüğünü belirle. Hata tarayıcıdan kaynaklanıyorsa üreticiye bildir (Firefox Bugzilla, WebKit, Chromium). Senin kodundansa sorunlu tarayıcı için ayrı bir kod yolu oluştur ve testi baştan yap.

Destek seviyeleri: A, B ve C

Seviye	Tarayıcılar	Yapılacak iş
A	Yaygın ve modern tarayıcılar	Kapsamlı test yapılır; tam destek verilir.
B	Eski ve sınırlı tarayıcılar	Test edilir; temel bilgi ve hizmetlere erişim sağlayan daha sade bir deneyim sunulur.
C	Nadir ya da bilinmeyen tarayıcılar	Test edilmez, çalışacağı varsayılır; önceden eklenen yedek çözümlerle tam site sunulur.

Örnek: Kuzey Amerika ve Batı Avrupa

A: Chrome, Firefox, Safari ve Edge'in son iki sürümü, iOS Safari, Android tarayıcıları; ayrıca erişilebilirlik testleri. **B:** Opera Mini. **C:** Opera ve az kullanılan diğer modern tarayıcılar.

Bu sınıflandırmayı Yahoo'nun "Graded Browser Support" yaklaşımı yaygınlaştırmıştır.

Hedef kitle ve test ölçütleri

Demografik tahmin

Kullanıcılar Kuzey Amerika ve Batı Avrupa'daysa çoğu Windows ya da Mac kullanır; telefonlarda iOS ve Android yaygındır. Ana tarayıcıların son sürümlerini test etmek yeterlidir.

Tarayıcı istatistikleri

Statcounter gibi siteler, bölgelere göre tarayıcı kullanım oranlarını gösterir.

Analiz araçları

En doğru veri, Google Analytics gibi araçlardan gelir (Kitle › Teknoloji). Matomo ve Open Web Analytics kendi sunucuna kurulabilir.

Test ölçütü örneği: soru işareti düğmesiyle açılan bilgi paneli

A ve B seviyesi

- Düğme fare, klavye ve dokunmatik ekranla çalışmalı.
- Basınca bilgi kutusu açılıp kapanmalı.
- Metin okunabilir olmalı.
- Ekran okuyucuyla metne ulaşılabilmeli.

Yalnızca A seviyesi

Panel akıcı bir animasyonla açılıp kapanmalı.

Ölçütlerin faydası

- Test adımlarını verir.
- Kullanıcı yönergesine dönüşür.
- Otomatik testin temelidir.

Veriler yanıltabilir: bir tarayıcıda kullanıcı görünmemesi, sitenin orada bozuk olmasından kaynaklanabilir.

Test ortamı: gerçek cihazlar, emülatörler, sanal makineler

Gerçek cihaz	Kurulu olması gereken tarayıcılar
Mac	Firefox, Chrome, Opera, Safari
Windows bilgisayar	Edge, Chrome, Firefox, Opera
Android telefon ve tablet	Chrome, Firefox, Opera Mini, Android'in kendi tarayıcısı
iOS telefon ve tablet	iOS Safari, Chrome, Firefox, Opera Mini
İsteğe bağlı	Linux bilgisayar (Linux'a özgü hatalar için), düşük donanımlı cihazlar (performans için)

Gerçek cihaz en doğru sonucu verir. Küçük ekipler çoğunlukla emülatör, sanal makine ve ticari servisleri birlikte kullanır.

Emülatör

• Bir cihazı ya da cihaz koşullarını yazılımla taklit eder.

• Firefox, Chrome ve Safari'nin duyarlı tasarım modları, ekran boyutunu hızlıca denemeyi sağlar.

• Android Studio emülatörü; Xcode ile gelen iOS Simulator (yalnızca Mac'te çalışır).

Sanal makine

• Bir işletim sistemini sanal diskte çalıştırır. VirtualBox ücretsizdir; Parallels ve VMware de kullanılır.

• Kurulum imajı ve lisans gerekir; disk en az 10 GB, tercihen 50 GB ve üzeri olmalı.

• Windows'ta aynı anda birden çok Edge sürümü kurulamadığı için her sürüm ayrı sanal makinede çalıştırılır.

Kullanıcı testi

Kullanıcı grubu

Arkadaşlar, aile, iş arkadaşları, öğrenciler ya da ücretli test kullanıcıları olabilir. Değişiklik yayına alınmadan önce bir geliştirme sunucusunda, adım adım yazılmış bir test senaryosuyla denenir.

Örnek senaryo

- Düğmeye fareyle tıkla, sayfayı yenile.
- Düğmeyi klavyeyle seç ve etkinleştir.
- Dokunmatik ekranda düğmeye dokun.
- Bilgi kutusu açılıp kapanıyor mu? Metin okunuyor mu? Animasyon akıcı mı?

Temiz test ortamı

- Eklentisiz ayrı bir tarayıcı profili kullan.
- Gizli pencere aç; çerezler ve geçici dosyalar sonucu etkilemesin.
- Düşük donanımlı bir telefonda da dene.
- Bazı sunucu ortamları değişikliği yalnızca belirli kullanıcılara açabilir (ör. Django Waffle Flags).

HTML ve CSS sorunları: doğrulama ve linter

Sorunların bir kısmı, HTML ve CSS basit göründüğü için yeterince özen gösterilmemesinden kaynaklanır: anlamsal olmayan işaretleme, tüm içeriğin JavaScript ile üretilmesi (erişilebilirliği ve performansı bozar), tarayıcılarda farklı desteklenen yeni özellikler ve duyarlı tasarım hataları.

Doğrulama

HTML'de etiketler kapatılmış ve doğru iç içe geçmiş olmalı, doctype bulunmalı, etiketler amacına uygun kullanılmalıdır. W3C Markup Validation Service ve W3C CSS Validator hataları listeler.

Linter

Hataların yanında kötü uygulamalar için de uyarı verir; sıkılığı ayarlanabilir. Çevrimiçi araçlar (ör. Dirty Markup) olsa da asıl kolaylık editör eklentileridir: SublimeLinter, Notepad++ linter, VS Code linter'ları.

Geliştirici araçları

CSS hatalarını bulmak için çok işe yarar; Firefox'ta uygulanmayan bildirimler üstü çizili ve uyarı simgesiyle görünür. HTML hataları burada kolay görünmez, çünkü tarayıcı bozuk işaretlemeyi kendisi düzeltir. HTML için doğrulayıcıyı kullan.

CSS çakışmaları: Üçüncü taraf kodlardaki sınıf adları seninkilerle çakışabilir. Bileşenine ayrı bir sınıf verip seçicileri o sınıfla başlat, örneğin .audio-player ul a

HTML ve CSS'te yedek davranış (fallback)

HTML'de

• Tanınmayan eleman, anlamsız satır içi eleman (span gibi) sayılır; adıyla biçimlendirilebilir.

• video, audio, picture, canvas gibi elemanlarda etiketlerin arasına yedek içerik yazılır.

• Desteklenmeyen input türleri (date, color…) type="text" gibi çalışır.

CSS'te

• Tarayıcı anlamadığı bildirimi hata vermeden yok sayar; aynısı media query ve @supports için de geçerlidir.

• Önce eski değeri, sonra modern değeri yaz; modern değer destekleniyorsa eskinin yerine geçer.

• Listedeki tek geçersiz seçici, bütün listeyi geçersiz kılar.

HTML

```html
<video controls>
  <source src="clip.webm" />
  <!-- yedek içerik -->
  <a href="clip.mp4">Videoyu indir</a>
</video>
```

CSS

```css
background-color: red;
background-color: rgb(255 0 0 / 90%);
```

CSS ön ekleri ve duyarlı tasarım

CSS ön ekleri

• Deneysel özellikler üreticilerin kendi ön ekleriyle sunulurdu: Firefox için **-moz-**, Chrome, Edge, Opera ve Safari için **-webkit-**.

• Üretim sitelerinde kullanılmamalıydı; haber verilmeden değişebilir ya da kaldırılabilir. Yalnızca -webkit- yazmak, sitenin diğer tarayıcılarda bozulmasına yol açar.

• Ön ekin gerçekten gerekli olup olmadığını MDN ve caniuse.com'dan kontrol et. **Standart sürümü, ön ekli sürümden sonra yaz.**

ön ekli ve standart sürüm

```
.masked {
  -webkit-mask-image: url(MDN.svg);
  mask-image: url(MDN.svg);
  -webkit-mask-size: 50%;
  mask-size: 50%;
}
```

Yerleşim

Masaüstü için hazırlanan düzen telefonda kötü görünür. Mobil düzen, media query'ler ve viewport etiketiyle oluşturulur.

Çözünürlük

Mobil cihazlara büyük görsel göndermek bant genişliğini boşa harcar; küçük görseller ise yüksek çözünürlüklü ekranda bulanık görünür. Çözüm: picture elemanı, srcset ve sizes nitelikleri.

Yardım kaynakları

MDN (aramaya "mdn" eklemek yeterli), caniuse.com ve Stack Overflow. Hata mesajını doğrudan aratmak da çoğu zaman işe yarar.

4 · ÖZELLİK TESPİTİ

Özellik tespiti (feature detection)

Nedir?

Tarayıcının bir özelliği destekleyip desteklemediğini kontrol edip buna göre farklı kod çalıştırmaktır. Tarayıcı adına bakmak (user agent) önerilmez; özelliğin kendisi kontrol edilir.

CSS'te @supports

Özellik desteğine göre kural uygular; JavaScript gerektirmediği için tercih edilir. JavaScript karşılığı CSS.supports()'tur.

JavaScript kalıpları

- Nesnenin üyesi var mı? ('in' ile)
- Elemanın özelliği var mı?
- Metodun dönüş değeri nedir?
- Atanan değer korunuyor mu?

JavaScript

```
if ("geolocation" in navigator) {
  // konumu haritada göster
} else {
  // statik harita sun
}
```

CSS

```
@supports (display: grid) {
  .kutu { display: grid; }
}
@supports not (display: grid) {
  /* yedek kurallar */
}
```

Otomasyon: görev çalıştırıcılar ve Gulp

Görev çalıştırıcılar

Grunt, Gulp ya da npm script'leri, derleme sırasında linting, kod küçültme, CSS'e ön ek ekleme gibi işleri otomatik yapar.

Tarayıcı otomasyonu

Selenium, kurulu tarayıcılarda testleri çalıştırır. Sauce Labs ve BrowserStack gibi servisler Selenium tabanlıdır ve kurulum gerektirmez.

Gulp ile başlamak

- npm install --global gulp-cli
- npm install --save-dev gulp
- gulpfile.mjs oluştur; package.json'a "type": "module" ekle
- Görevler: html-tidy, csslint + autoprefixer, jshint + babel
- gulp.watch() kaydedilen dosyada görevleri yeniden çalıştırır

gulpfile.mjs

```
export function css() {
  return gulp
    .src("src/style.css")
    .pipe(csslint())
    .pipe(autoprefixer({ cascade: false }))
    .pipe(gulp.dest("build"));
}
```

5 · OTOMASYON

Ticari test servisleri

Servis	Elle (manuel) test	API
BrowserStack	Live panelinde platform, cihaz ve tarayıcı seçilir; geliştirici araçları, ağ yavaşlatma, ekran okuyucu ve ekran görüntüsü sunulur.	REST API ile plan, build ve oturum bilgilerine ulaşılır.
Sauce Labs	Test edilecek URL ve tarayıcı/işletim sistemi seçilir; uzaktan oturum, ekran görüntüsü, video ve günlük kayıtları alınır.	REST API (node-saucelabs ile); testlerin başarılı ya da başarısız olduğu bilgisi kaydedilebilir.
TestingBot	Live Web Testing bölümünde URL ve tarayıcı/işletim sistemi seçilir; çözünürlük değiştirilebilir, ekran görüntüsü alınabilir.	REST API (testingbot-api istemcisi).

Servis, seçilen işletim sistemi ve tarayıcıyla bir sanal makine başlatır; sonuçlar ekran görüntüsü, video ve günlük olarak sunulur. Bunlar tek seçenek değildir, yalnızca başlamak için kolay olanlardır.

5 · OTOMASYON

Selenium ve WebDriver kurulumu

1 · Proje

npm init ile proje oluştur, ardından npm install selenium-webdriver komutunu çalıştır.

2 · Sürücüler

Firefox için GeckoDriver, Chrome için ChromeDriver'ı indir.

3 · PATH

Sürücülerin bulunduğu klasörün yolunu PATH değişkenine ekle (dosyanın değil, klasörün yolunu).

4 · Çalıştırma

node duck_test komutuyla testi başlat.

duck_test.js

```
const { Builder, Browser, By, Key }
  = require("selenium-webdriver");

(async function example() {
  const driver = await new Builder()
    .forBrowser(Browser.FIREFOX).build();
  try {
    await driver.get("https://duckduckgo.com/");
    await driver.findElement(By.name("q"))
      .sendKeys("webdriver", Key.RETURN);
  } finally { await driver.quit(); }
})();
```

WebDriver söz dizimi ve iyi test uygulamaları

Adım	Kod	Açıklama
Sürücüyü oluşturma	new Builder().forBrowser(Browser.FIREFOX).build()	Tarayıcıyı seçer; istenirse sürüm ve işletim sistemi de belirtilir.
Sayfayı açma	driver.get(url)	http(s) ya da file:// adresi kullanılabilir.
Eleman bulma ve kullanma	driver.findElement(By.css("button")).click()	By.id, By.name, By.css ile seçilir; sendKeys() ve getText() kullanılabilir.
Bekleme	driver.sleep(ms) driver.wait(until…)	sleep() sabit süre bekler, wait() bir koşul gerçekleşene kadar bekler.
Kapatma	driver.quit()	Tarayıcıyı kapatır, kaynakları serbest bırakır.

İyi test için
- Kalıcı ID ya da konum gibi değişmeyecek seçiciler kullan.
- Her test yalnızca tek bir şeyi denetlesin.
- Testler birbirine bağımlı olmasın.

Sonuçları raporlama
console.log yerine Mocha ve Chai gibi araçlar kullanılabilir. Çalıştırma: npx mocha --no-timeouts

Uzak test ve CI
usingServer() ile BrowserStack, Sauce Labs ya da kendi Selenium sunucuna bağlanılabilir (localhost:4444). CI araçlarıyla testler geçmeden değişiklik koda alınmaz (ör. Travis CI).

Ders özeti

Giriş	Sorunları erken aşamada yakala; testi sürecin parçası yap.
Test stratejileri	Hedef kitleyi belirle; A, B, C seviyeleri ve test ölçütleriyle çalış.
HTML ve CSS	Doğrula, yedek davranışlardan yararlan, ön ekleri doğru kullan.
Özellik tespiti	Tarayıcının adına değil, özelliğin desteğine bak.
Otomasyon	Tekrarlanan testleri Gulp, ticari servisler ve Selenium'a bırak.

Kaynak: developer.mozilla.org › Learn web development › Extensions › Testing